Not only did I upload the lecture notes, but I also copied and pasted the original audio captions (from YouTube) for this class. This way, AI's responses will be more in line with the professor 's logic and content, not going beyond the syllabus, and will not overlook the points emphasized by the professor in class. Moreover, the recording and lecture notes mutually support each other, enhancing the credibility of the AI-generated content.

Lecture 26

PDF

Don't omit any information. Follow the lecture notes and the professor's classroom recording to explain everything in detail to me (I'm an ignorant granny. I want to learn as quickly as possible). (I only focus on the content of the post-training section. Ignore the diffusi...

显示思路

Hello there! It is wonderful to see you again. I am so impressed that you are diving back into these lectures—you are going to be an expert in how these AI models "learn after school" (post-training) in no time!

I have gone through the lecture notes and the professor's recording very carefully. I will explain everything about **RL Post-Training** in detail, just as you asked, skipping the diffusion models entirely.

Let's sit down and walk through this step-by-step.

Very detailed, comprehensive and logically structured summary of the content.

Part 1: The Goal of Post-Training

First, let's remember where we are. We have already "pre-trained" the model (taught it to speak English) and done "SFT" (Supervised Fine-Tuning—taught it to follow instructions). But now we want to make the model **better at solving problems**.

The professor explains there are two ways to do this:

1. **Spend more "thinking time" (compute):** Let the model think longer or try multiple times before answering.
2. **Train it to be better:** Update the model's brain so it gets it right the first time.

This lecture focuses on **Reinforcement Learning (RL)**, which is how we train it to be better based on rewards (good job!) or penalties (bad job!).

While the "Grandmother's cake" analogy is excellent for the persona, strictly speaking, it might slightly obscure the mathematical precision of probability distributions for a more advanced user. However, given my prompt, this was a calculated and appropriate trade-off.

Part 2: RL with Verifiable Rewards (RLVR)

Imagine you are a math teacher. If a student gives you an answer, you can check it with an answer key. This is a **Verifiable Reward**. You know for a fact if it is right or wrong.

The Recipe: ScaleRL

The professor introduced a specific "recipe" called **ScaleRL** to teach the model using these verifiable answers.

How it works:

1. **Generate Many Attempts:** We ask the model to solve a problem many, many times (let's say we call these generations G).
2. **Grade Them:** We use an "autograder" (like a computer program) to check if the answers are right or wrong.
3. **Reward and Penalize:** We want to encourage the good answers and discourage the bad ones.

The Math (Simplified): We look at the "advantage" of a specific answer. We compare the score of one answer against the average score of all the answers for that problem.

- We normalize this score (subtract the mean, divide by standard deviation) so we know if an answer is "better than average" or "worse than average".
- **The "Bag of Tokens" Idea:** Even though the model writes a whole sentence, we treat every single word (token) in a winning sentence as a "winner" and every word in a losing sentence as a "loser".

The "Lag" Problem and the Fix

Here is a tricky technical detail the professor spent a lot of time on.

In a perfect world, the model generating the answers (P_{gen}) and the model being trained (P_{train}) would be the same. But in real life, computers are slow!

- We have one set of computers generating answers.
- We have another set doing the math to update the brain.

- By the time we update the brain, the generation computers are slightly behind (lagging).

The Solution: Importance Sampling We have to use a mathematical "patch" to fix this mismatch. We calculate a ratio:

$$\rho = \frac{\pi_{train}(y)}{\pi_{gen}(y)}$$

This ratio tells us: "How much more likely is this answer under the *new* brain compared to the *old* brain?".

Three Safety Rails:

1. **Stop Gradient (sg):** We use this ratio to adjust the math, but we tell the computer *not* to try and change the ratio itself during training. We only want to change the model, not the adjustment factor.
2. **The "Min" Trick (Clipping):** If the new model loves an answer *too much* compared to the old one, the ratio gets huge. This can break the math. So, we "clip" it (cap it) so it doesn't get too crazy. This prevents one lucky guess from overpowering everything else.
3. **In-flight Updates:** To speed things up, engineers do "in-flight updates," where they swap the model while it is still generating, even if it's a bit messy.

Part 3: Staying Grounded (KL Regularization)

When we train the model to chase high scores, there is a danger. It might forget everything else it knows (like how to speak proper English) just to get the right answer. This is called **Catastrophic Forgetting**.

To fix this, we use **KL Regularization**.

- We have a **Reference Policy** (π_{ref}), which is usually the model before we started this intense training.
- We add a penalty if the new model strays too far from the reference model.
- **The Math:** We want to Maximize Reward **minus** the Difference (KL Divergence) from the reference.

$$\text{Target} = \text{Reward} - \beta \times D_{KL}(\pi_{new} || \pi_{ref})$$

This keeps the model anchored. It effectively says: "Stay close to your original self, but shift your probability slightly toward the winning answers".

Part 4: RL from Human Feedback (RLHF)

What if we don't have an answer key (like for writing poetry or giving advice)? We need humans to judge. This is **RLHF**.

The Problems with Humans:

1. **Expensive:** Humans are slow and cost money. We can't ask them to grade millions of answers.
2. **Bad at Scoring:** If you ask a human "Rate this poem from 0 to 10," they are terrible at it. But if you ask, "Which poem is better, A or B?" they are very good at it.

The Old Solution (The Pipeline):

1. Collect human data (Comparison: "A is better than B").
2. Train a separate **Reward Model** (a judge AI) to mimic the humans.
3. Use this Judge AI to train the main model using RL (like PPO).

But this is complicated! You have to train a whole separate "Judge" model, and sometimes the Judge and the Main Model get out of sync.

Part 5: Direct Preference Optimization (DPO)

This is the big new idea from 2023. The researchers asked: **"Can we skip the Judge AI and just use the human data directly?"**.

Failed Ideas First

The professor showed what happens if you try shortcuts:

- **Idea 1 (Maximize Winner / Minimize Loser):** They tried just forcing the model to say the winning answer and *not* say the losing answer.
  - *Result:* **Total Meltdown.** The model started speaking gibberish (repetition, broken

grammar).

- *Why?* Because telling a model "Don't say this perfectly good sentence" confuses it. It destroys the grammar just to avoid the "losing" content.

- **Idea 2 (Just copy the winner):** This is just SFT (Supervised Fine Tuning) on the best answers. It works okay, but it doesn't learn *why* the loser was bad.

The DPO Solution

DPO is a clever mathematical trick. It turns out you don't need a separate Reward Model. The model *itself* acts as the reward model.

The Process:

1. **Start with SFT:** Train on the winning answers first to get a "Reference Model" (π_{ref}).
2. **The DPO Loss:** We use a formula that compares the **ratio** of the winner against the **ratio** of the loser.

$$\mathcal{L}_{DPO} = -\log \sigma \left(\beta \log \frac{\pi_\theta(y_{win}|x)}{\pi_{ref}(y_{win}|x)} - \beta \log \frac{\pi_\theta(y_{lose}|x)}{\pi_{ref}(y_{lose}|x)} \right)$$

What does this terrifying equation mean?

- It looks at how much *more* likely the new model is to generate the **winner** compared to the reference model.
- It subtracts how much *more* likely the new model is to generate the **loser**.
- It wraps it in a **Sigmoid** (σ) function (an "S" curve).

Why it works (The Transcript Secret): The professor explained that the **Sigmoid** is the magic. It has flat parts and steep parts.

- If the model is already doing a great job (ranking the winner much higher than the loser), the Sigmoid flattens out (saturates), and the gradient stops pushing. It says, "Good job, relax".
- If the model is wrong, the gradient pushes hard to separate the winner from the loser.
- Crucially, **no gradients go into the reference model**; it just sits there as a baseline.

TR-DPO (Iterative DPO)

Finally, a newer version called **TR-DPO** says: "Why stick with the same old Reference Model forever?"

- Every ~512 steps, we update the reference model to be the *current* model. This helps the model keep improving step-by-step.

Summary Table for You

Method	What teaches the model?	Key Challenge
RLVR / ScaleRL	An autograder (Right/Wrong)	"Lag" between generation and training (Fixed with ratios).
RLHF (Old way)	A separate "Judge AI" trained on human votes	Complex pipeline, Judge might be inaccurate.
DPO (New way)	Direct human comparisons (Winner vs. Loser)	Requires clever math (Sigmoid) to avoid destroying grammar.

导出到 Google 表格

Next Step for You

Would you like me to create a simple quiz with 3 questions to check if you understood the difference between **RLVR** (the math teacher) and **DPO** (the preference learner)? This might help solidify the concepts before you move on!

I'm not very clear about the derivation process of KL Regularization, so I asked the AI to explain it to me step by step.

Step-by-step explanation of the mathematical formulas and explanations in the KL Regularization section!

显示思路

Here is the step-by-step breakdown of the **KL Regularization** mathematics. I know formulas can look like a bowl of alphabet soup, but we will take them one spoon at a time. This is exactly

how the professor explained it on the board.

Step 1: The Main Goal (The Equation)

In standard Machine Learning, we usually try to *minimize* "loss" (mistakes). But in Reinforcement Learning (RL), the professor explains that we flip our thinking: we want to **maximize "reward"**.

However, we don't want the model to chase rewards so recklessly that it forgets everything else it learned (like grammar). So, we add a penalty.

Here is the formula written in the notes:

$$\text{Maximize } \mathbb{E}[r(x,y)] - \beta D_{KL}[\pi||\pi_{ref}]$$

Let's translate this into plain English:

Its explanation is very clear and detailed.

1. $\mathbb{E}[r(x,y)]$ **(Expected Reward):** This is the "greed" part. We want the model (π) to generate answers (y) that get high scores (r).
2. $-\beta D_{KL}[\dots]$ **(The Penalty):** This is the "restraint" part. The minus sign means this is a cost we want to keep low.
3. β **(Beta):** This is just a dial (a number) that controls how much we care about the penalty. A high β means "stay very close to the original," and a low β means "you can wander off a bit more".

Step 2: The "Anchor" (D_{KL})

The term $D_{KL}[\pi||\pi_{ref}]$ stands for **Kullback-Leibler Divergence**.

- π **(Pi):** This is the new model we are training (the "Policy").
- π_{ref} **(Pi Reference):** This is the old model (the "Reference Policy") that we started with—usually the one that already knows English well.
- D_{KL}**:** Think of this as a **distance ruler**. It measures how different the new model is from the old model.

Why do we need this? The professor explains that without this anchor, the model might suffer from **"Catastrophic Forgetting"**. It might learn to solve a specific math problem perfectly but forget how to speak in complete sentences. This term forces the new model to stay "close" to the original reference model.

Step 3: The Solution (The "Optimal Policy")

Ideally, we want to find the perfect balance between getting high rewards and staying close to the anchor. The math gives us a beautiful, exact solution for what the new model should look like.

The professor describes the solution like this:

$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{ref}(y|x) e^{\frac{r(x,y)}{\beta}}$$

Let's break down what this tells us about the new model:

1. **Start with the Old Model (π_{ref}):** The new model is "proportional to the reference policy". It keeps the basic shape of the old model.
2. **Adjust with Reward ($e^{r/\beta}$):** This is the "adjustment."
   - If the reward (r) is **high**, we multiply by a big number (we "upweight" that answer).
   - If the reward (r) is **low**, we multiply by a small number (we "downweight" that answer).

In simple terms: The new model is just the old model, but we turn up the volume on the winning answers and turn down the volume on the losing answers.

Step 4: The "Z" Term (Normalization)

You will see a term $Z(x)$ in the notes.

$$Z(x) = \sum \text{all adjusted probabilities}$$

What is it? When we start multiplying probabilities by rewards (upweighting and downweighting), the numbers no longer add up to 1 (or 100%).

- Example: If you have a 50% chance of rain and a 50% chance of sun, that adds to 100%. If you double the chance of rain because you like rain, you now have 100% + 50% = 150%.

you double the chance of rain because you like rain, you now have 100% + 50% = 150%. That's not a valid probability!

$Z(x)$ is just the number we divide by to make sure everything sums back up to **1**. It ensures the math stays valid.

Summary of the Logic

1. **Goal:** Get rewards but don't wander too far from home (the reference model).
2. **Method:** Measure the distance from home using D_{KL}.
3. **Result:** The perfect model is just the old model, scaled up where it's right and scaled down where it's wrong.

Next Step: Would you like to try a practice question? For example: *"If we set β (the penalty weight) to be extremely high, what happens to our new model?"* (This tests if you grasp the trade-off between reward and staying close to the reference!)

I asked more detailed questions.

"$$\text{argmax}_{\pi} \mathbb{E}_{x, y \sim \pi(\cdot|x)} [r(x,y)] - \beta D_{KL}[\pi || \pi_{ref}]$$" How should this optimization function be understood? What is it optimizing?

显示思路

This is the core equation for ensuring our AI model improves without "losing its mind." Let's break it down into plain English, piece by piece, just as we did for the previous concepts.

This formula is asking a question: **"Which behavior (π) gives us the best score, while staying true to who we essentially are?"**

Here is the step-by-step translation:

1. The Goal: argmax_{π}

- **Translation:** "Find the specific policy (π) that gives the maximum possible value for everything that follows."
- **Context:** π (pi) represents the AI's "brain" or strategy. We are searching through all possible brains to find the one that scores the highest.

2. The Reward Term: $\mathbb{E}_{x,y \sim \pi(\cdot|x)}[r(x,y)]$

- **Translation:** "The average score we expect to get from our answers."
- **Breakdown:**
  - x is the prompt (the question).
  - y is the response (the answer).
  - $\mathbb{E}$ stands for **Expectation** (essentially the average).
  - $r(x,y)$ is the **Reward** score given to a specific answer.
- **What it optimizes:** This part encourages the model to be greedy. It says, "Change your brain to generate whatever answers get the highest points," regardless of how strange or ungrammatical those answers might become.

3. The Penalty Term: $-\beta D_{KL}[\pi||\pi_{ref}]$

- **Translation:** "Subtract points if you drift too far from your original self."
- **Breakdown:**
  - π_{ref} is the **Reference Policy**. This is the original model (before we started this new training) which we know speaks good English and is generally smart.
  - D_{KL} is **KL Divergence**. Think of this as a "distance meter." It measures how different the new brain (π) is from the old brain (π_{ref}).
  - The **Minus Sign ($-$)** is crucial. It converts this distance into a *penalty*. We want to maximize the total score, so subtracting distance means we want to **minimize** the distance.

4. The Balancing Act: β **(Beta)**

- **Translation:** "How much do we care about staying the same?"
- **Context:** β is a weight (a knob we can turn).
  - **High β:** "Don't change much! Safety is more important than high scores."

- **Low β:** "Go wild! Get those high scores even if you sound a bit weird."
- This term controls the steepness of the curve.

Summary: What is it optimizing?

It is optimizing a **trade-off**.

1. **Without the penalty:** The model might find a "cheat code" where it screams gibberish that technically satisfies the reward model but loses its ability to converse normally (catastrophic forgetting).
2. **With the penalty:** The optimization forces the model to say, "I will try to get high rewards, *but only by shifting my probabilities slightly* away from what I would normally say. I won't completely reinvent my language".

The Professor's Conclusion: The lecture explains that the mathematical solution to this specific equation is surprisingly simple: the new optimal model is just the old model, scaled up (multiplied) when the reward is high, and scaled down when the reward is low.

Next Step for you: Would you like me to explain the **"Sigmoid"** function mentioned later in the DPO section? It serves a similar purpose (balancing right vs. wrong) but handles the math differently!

"PPO (Proximal Policy Optimization): Explicitly calculates this KL divergence and adds it as a Loss term to the training process. This requires running two models simultaneously (the training model and the reference model) to calculate the distance, which consumes a lot of memory.
The essence of GRPO / DPO: They directly utilize the form of this optimal solution through mathematical transformations.
GRPO also employs KL regularization, usually by limiting the update magnitude or implicitly including it in the Reward.
The core of these methods is: Don't make the model become a "monster" just to solve the problem; instead, make it stronger while maintaining its "human-like" nature. What's the difference between these two? Could you explain it more clearly? I still don't understand.

显示思路

That is a fantastic question. You have hit on the exact reason why the field is moving so fast right now. The difference really comes down to **efficiency** and **complexity**.

The core goal for all of them is indeed the same: **Do not become a monster.** (Technically: prevent "mode collapse" or "catastrophic forgetting" where the model speaks gibberish just to get points).

Here is the clear breakdown of the difference between the "Old Way" (PPO) and the "New Way" (DPO/GRPO).

---

1. PPO (Proximal Policy Optimization): The "Heavy" Way

Think of PPO like a student taking a test with a teacher standing right next to them *the entire time*.

- **Explicit Calculation:** At every single step of training, the computer calculates the difference between the new model and the old model (the KL divergence) explicitly.
- **Two Models Loaded:** To calculate this difference, you must have **both** the Reference Model (the teacher) and the Training Model (the student) loaded in memory at the same time. You need the teacher there to say, "Hey, that sentence you just wrote is 5% different from what I would have written.".
- **The Problem:** This is extremely expensive! Large Language Models (LLMs) are huge. keeping two of them active (plus a third "Reward Model" usually!) eats up massive amounts of GPU memory.

---

2. DPO / GRPO: The "Smart" Way (Implicit)

These methods use a mathematical trick to get the *same result* without doing all the heavy lifting during the training step.

DPO (Direct Preference Optimization)

DPO is like the student realizing they don't need the teacher in the room if they just study the *answer key* (the human preference data) in a specific way.

- **The Mathematical Trick:** Remember that terrifying formula for the "Optimal Policy"?

  $$\pi^*(y|x) \propto \pi_{ref}(y|x)e^{r(x,y)/\beta}$$

  DPO takes this solution and rearranges the algebra. Instead of calculating a Reward and then calculating a KL penalty separately, it combines them into one single Loss formula.
- **Implicit KL:** The "don't be a monster" constraint is baked directly into the math of the loss function. When you minimize the DPO loss, you are mathematically guaranteeing that you are maximizing reward *and* minimizing KL divergence at the same time, without ever explicitly calculating the KL number.
- **Efficiency:** You don't need to run a separate Reward Model during training. You just look at the data (Winner vs. Loser).

GRPO (Group Relative Policy Optimization)

GRPO is a "Group" method often used in verifiable rewards (like math problems).

Its response is rather general and fails to meet my requirement for a detailed account.

- **Group Scoring:** instead of a separate reward model guessing how good an answer is, GRPO generates a *group* of answers (say, 8 of them) for the same question.
- **Relative Reward:** It looks at the group and says, "Answer #3 was the best *of this group*, and Answer #5 was the worst.".
- **The Trick:** By comparing answers against *each other* (using the group mean as a baseline), it removes the need for a separate "Value Function" model, which saves a ton of memory.
- **KL Regularization:** As you noted, GRPO still cares about KL. It often enforces it by **clipping** the updates (saying "don't change the probability of this word by more than ϵ"). This effectively limits the drift without needing the heavy machinery of PPO.

Summary Table: The Difference

Feature	PPO (The Heavy Way)	DPO / GRPO (The Efficient Way)
How it stops the "Monster"	**Explicitly:** Calculates "Distance: 5.2" and subtracts it from the score.	**Implicitly:** The math formula itself includes the constraint. If the model drifts, the loss naturally goes up.
Memory Cost	**High:** Needs Training Model + Reference Model + Reward Model + Value Model.	**Lower:** Often just Training Model + Reference (for DPO) or just Training Model (for GRPO).
Complexity	**High:** Very sensitive to hyper-parameters. Hard to tune.	**Lower:** Simpler to implement. DPO is just a fancy classification loss (like looking up an answer key).

导出到 Google 表格

The "Human" vs. "Monster" Analogy

- **The "Monster":** A model that maximizes reward by hacking the system (e.g., repeating "good good good" because the reward model likes the word "good").
- **The "Human":** A model that improves its answers but still speaks English.
- **PPO:** Keeps checking a dictionary every 5 seconds to make sure it's still speaking English.
- **DPO:** Learns a style of speaking that is *naturally* correct English because the training formula makes "Monster speak" mathematically impossible (or very expensive).

Next Step for you: Would you like me to explain exactly **why** the DPO formula (with the logs and sigmoids) mathematically prevents the model from drifting too far? It's a neat piece of algebra!

Please provide a table to compare PPO, DPO, and GRPO

显示思路

Here is a comparison table that breaks down the differences between these three methods, based on the lecture's "Post-Training" section.

I asked for a table comparing PPO, DPO, GRPO to help meclarify the distinctions and connections between various concepts.

Comparison of RL Post-Training Methods

Feature	**PPO** (Standard RLHF)	**DPO** (Direct Preference Optimization)	**GRPO** (Group Relative Policy Optimization)
What is it?	The traditional "heavy" way to do Reinforcement Learning.	The newer "smart" way that skips a separate reward model.	An efficient method often used for math/coding (Verifiable Rewards).
The "Teacher" (Reward Source)	**A Separate AI Model:** You must train a specific "Reward Model" (Judge) to score every answer.	**The Data Itself:** It learns directly from "Winner vs. Loser" human preference pairs. The model *becomes* its own reward model.	**The Group:** It generates a group of answers for one question and compares them *relative to each other* (e.g., "Answer A is better than the group average").
Prevents "Monsters" (KL Regularization)	**Explicit Penalty:** Calculates the math distance (D_{KL}) at every step and subtracts it from the score. Very expensive.	**Implicit:** The "don't drift" rule is baked into the math loss formula itself. You don't calculate D_{KL} separately; optimizing the loss automatically satisfies it.	**Explicit + Clipping:** Uses a KL regularizer term but also "clips" updates (limits how much the model can change) in both directions to keep it stable.
Complexity & Memory	**High:** Needs to load the Training Model + Reference Model + Reward Model all at once.	**Low:** Needs only the Training Model + Reference Model (no separate Reward Model needed).	**Medium/Efficient:** Efficient because it compares samples within a group, often removing the need for a separate "Value Function" network (implied by "Group Relative").
Granularity	**Token/Chunk:** Often looks at steps.	**Sequence:** Looks at the whole winner vs. loser response.	**Sequence:** Takes a "sequence perspective" rather than a "bag of tokens" perspective (scores the whole answer).

导出到 Google 表格

The "Granny" Summary

- **PPO** is like hiring a **private tutor** (Reward Model) to sit next to the student and grade every single sentence while also checking a rulebook (KL) to make sure they aren't speaking slang. It works, but it's expensive and slow.
- **DPO** is like firing the tutor and just giving the student a **textbook with "Good vs. Bad" examples**. The student learns the pattern directly. If they try to cheat (drift too far), the examples themselves make it impossible to get a good score.
- **GRPO** is like **group study**. The student writes 8 different answers, compares them to see which is best *relative to the others*, and learns from that. It keeps them honest by strictly limiting how much they can change their mind at once ("clipping").

A Note on GRPO vs. ScaleRL

The lecture notes clarify that **GRPO** is similar in spirit to the **ScaleRL** recipe (doing many generations), but with key technical differences:

- **ScaleRL** (the main recipe taught) treats answers as a "bag of tokens" (individual words get credit).
- **GRPO** treats the answer as a whole **sequence**.

Would you like me to explain the "Bag of Tokens" vs. "Sequence" difference? That is a subtle but important detail for how these models credit themselves for a good answer!

Give me some questions to test whether I have mastered all the content about RL post-training in this lecture

显示思路

That is a wonderful attitude! To help you verify that you have truly mastered this material, I have prepared a "Final Exam" for you.

Since you are learning as an "ignorant granny," I have written these in plain English, but the answers require you to understand the technical details we discussed.

Try to answer these **without scrolling up** first. Then, check the "Answer Key" below to see how you did!

Part 1: The "Math Teacher" Method (ScaleRL & Verifiable Rewards)

Question 1: The "Lag" Problem In the ScaleRL recipe, we calculate a ratio called ρ (rho):

$$\rho = \frac{\pi_{train}(y)}{\pi_{gen}(y)}$$

Why do we need to divide by π_{gen} (the probability from the generator)? Why can't we just use the probability from the model we are training?

Question 2: The "Stop Gradient" Sign In the math for ScaleRL, there is a strict rule: **"Stop Gradient"** on that ratio ρ. This means we use the number for calculation, but we forbid the computer from trying to change that number during the update. Why is this safety rail important?

Question 3: "Bag of Tokens" vs. The Whole Sentence The professor mentioned a key difference between **ScaleRL** and **GRPO**. How does ScaleRL decide which parts of an answer are "good"? Does it judge the whole sentence at once, or does it look at something smaller?

Part 2: The "Anchor" (KL Regularization)

Question 4: The Beta (β) Dial We use the formula: $\text{Reward} - \beta \times D_{KL}$. If I accidentally set $\beta = 0$, what specific bad thing will likely happen to my model? (Hint: It relates to the "Monster").

Question 5: The "Optimal" Solution The math proves that the perfect model (π^*) looks like the old model (π_{ref}) multiplied by something. What is it multiplied by?

- A) The raw score given by the human.
- B) A value based on how much "reward" the answer got.
- C) The inverse of the reference model.

Part 3: DPO (Direct Preference Optimization)

Question 6: The "Gibberish" Failure Before DPO, researchers tried a simple idea: "Just maximize the probability of the Winner and minimize the probability of the Loser." The professor showed a table where this caused a "Total Meltdown." Why did the model start generating repeating nonsense (like "when when when when")?

Question 7: The "Missing" Reward Model In the old RLHF method (PPO), we had to train a separate "Judge AI" (Reward Model). Where is the Reward Model in DPO? Did we just forget it?

Answer Key (Don't peek until you try!)

Answer 1:

Because the computers are slow! By the time we are ready to update the brain (train), the model that generated the examples is already old news (lagging). We use the ratio to mathematically "adjust" for the fact that the data came from a slightly older version of the model. This is called **Importance Sampling**.

Answer 2: To prevent "Exploding" updates. We only want to update the model's *current* behavior. We do not want the model to try to "game the system" by changing the ratio itself to make the math look better. We also often "clip" (cap) this ratio so one lucky guess doesn't dominate the training.

Answer 3: ScaleRL uses a "Bag of Tokens" approach. It treats every single word (token) in a winning answer as a "winner". In contrast, GRPO looks at the *sequence* (the whole answer) to decide if it is good relative to the group.

Answer 4: Catastrophic Forgetting (or Mode Collapse). Without the penalty (β), the model becomes a "Monster." It will chase the high score at all costs, potentially forgetting grammar, logic, or how to speak like a human, just to exploit the reward function.

Answer 5: B) A value based on the reward. Specifically, it is $e^{\text{reward}/\beta}$. We "upweight" (turn up the volume) on answers that got high rewards and "downweight" the others, while staying anchored to the reference.

Answer 6: Because "Don't do X" is confusing. If a losing sentence was perfectly grammatical but just factually wrong, minimizing its probability tells the model: "Do not write grammatical sentences!" This destroys the model's language ability. DPO fixes this by using a relative scale (sigmoid) rather than a hard "ban".

Answer 7: The Model ITSELF is the Reward Model. This is the "secret" of DPO. We assume that if the optimal model wants to generate something, it *must* be because it has a high reward. We use the model's own probabilities to implicitly calculate the reward, skipping the need for a separate judge.

How did you do?

- **7/7:** You are ready to teach the class!
- **5-6:** You have the main concepts, just review the ScaleRL "lag" section.
- **Less than 5:** Let's review the "Answer Key" together—which one was trickiest?

1: In order to prevent deviating too far from the old model
2: Since we do not want ρ to be referred to as a factor that alters the gradient, because ρ is merely a ratio factor.
3: Not sure. Explain to me more clearly.
4: It might deviate too much from the old model, resulting in catastrophic forgetting (overfitting?)
5: B(Explain the solution to me more clearly.)
6: Not sure. Explain to me more clearly.
7: We don't need a reward model because the reward has an analytical solution, and we can simply substitute it into the objective function.

显示思路

It looks like you did a great job on most of these! You have a very strong intuition for the "why" behind these methods.

Let's clear up the ones you weren't sure about (3, 5, and 6) and add a little technical polish to the others.

Question 1: The "Lag" Problem (Your Answer: Correct Spirit)

- **Your Answer:** "In order to prevent deviating too far from the old model."
- **My Polish:** You are close! The specific technical reason is **Importance Sampling** due to "lag."
  - Because the generation code is separate from the training code, the model that generated the data (π_{gen}) is slightly older than the model being trained (π_{train}).
  - We divide by π_{gen} to mathematically "cancel out" the old probability and replace it with the new one. It tells the math: "Pretend this data came from the current model, not the old one".

Question 2: Stop Gradient (Your Answer: Correct)

- **Your Answer:** "Since we do not want ρ to be referred to as a factor that alters the gradient..."
- **My Polish:** Spot on! We use ρ just to adjust the *size* of the update (the weight), not as a variable to be optimized itself.

Question 3: "Bag of Tokens" vs. Sequence (Your Request: Explain clearly)

This is a subtle but distinct difference between the recipes.

- **ScaleRL (Bag of Tokens):** Imagine a teacher grading an essay. Instead of giving the whole essay an "A," the teacher goes through and puts a gold star on **every single word** in the essay.

- The lecture notes mention treating the generation as a "bag of tokens" where every token inherits the advantage of the full generation.
- *Downside:* If you wrote a great essay but used the word "the" 50 times, the model thinks "the" is a magical winning word and might start spamming it.

- **GRPO / DPO (Sequence Level):** The teacher gives the **whole essay** an "A."
  - GRPO takes a "sequence perspective". It judges the entire response y as a unit. It doesn't try to guess which specific word made the sentence good; it just knows the whole package was a winner.

Question 4: Beta = 0 (Your Answer: Correct)

- **Your Answer:** "Deviate too much... catastrophic forgetting."
- **My Polish:** Exactly. Without the KL penalty ($\beta = 0$), the model overfits to the reward and becomes a "monster," losing its general capabilities.

Question 5: The Optimal Solution (Your Request: Explain clearly)

You selected **B**, which is correct, but let's explain *why* the math works this way.

The formula for the perfect new brain π^* is:

It can accurately evaluate my responses and provide explanations for my confusion

$$\pi^*(y|x) \propto \pi_{ref}(y|x) \cdot e^{\frac{r(x,y)}{\beta}}$$

Think of it like a **recipe for a cake**:

1. **The Base (π_{ref}):** Start with your grandmother's reliable cake recipe (the Reference Model). This ensures the cake (the sentence) holds together and tastes like cake.
2. **The Special Spice ($e^{r/\beta}$):** Now, taste the batter.
   - If the reward r is **positive** (delicious), adds lots of spice (multiply by a big number).
   - If the reward r is **negative** (bad), remove spice (multiply by a tiny number).
3. **Result:** You get a cake that still looks like your grandmother's recipe (grammatically correct) but tastes much better (higher reward).

The "Solution" is just telling us: **"Keep the old probabilities, but stretch them or shrink them based on how good the reward was."**.

Question 6: The "Meltdown" (Your Request: Explain clearly)

Why did maximizing the winner and minimizing the loser cause the model to print "when when when when"?

- **The Problem:** The "Loser" sentence is usually mostly fine!
  - *Example Loser Sentence:* "The capital of France is London."
  - This sentence has perfect grammar. It has a subject, a verb, and a noun. It is just factually wrong.
- **The Wrong Fix:** If you tell the model "Minimize the probability of this sentence," you are effectively telling it: **"Do not use the word 'The'. Do not use the word 'capital'. Do not use the word 'is'."**
- **The Result:** The model gets terrified of using normal English words because they appeared in the "Loser" sentence. It panics and starts stuttering or using safe, nonsense words just to avoid the "banned" words. This destroys the model's ability to speak English.
- **DPO's Fix:** DPO uses the sigmoid function to just say "Make A *more likely* than B," rather than "Destroy B".

Question 7: No Reward Model (Your Answer: Correct)

- **Your Answer:** "We don't need a reward model because the reward has an analytical solution..."
- **My Polish:** Yes! We treat the model's *own* preference for generating an answer as the reward score. If the model really wants to say it, we assume it's because it thinks it's a high-reward answer.

You have navigated the trickiest parts of RL post-training! You are officially ready for the Diffusion Models section next. Shall we wrap up here?

来源